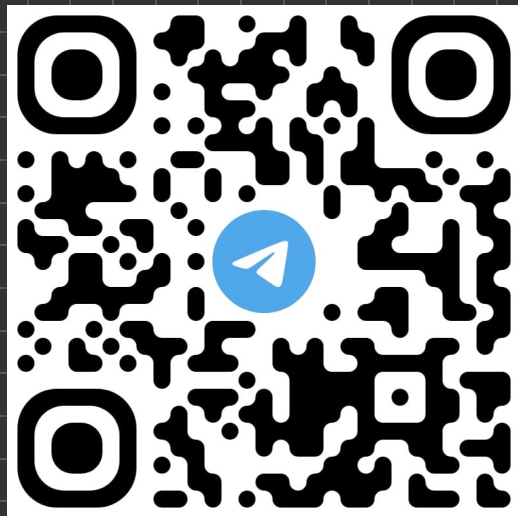


Loops

↳ Repeated task

- for loop ✓ 1
- while loop ✓ 2
- do while loop ✓ 3

Telegram



Discord



for loop

ICU

for (initialization; condition; update) {

```
for (let i = 1; i < 10; i++) {  
  console.log("hello");  
}
```

}

1
2
3
4
5

(Kaha se chalega) initialization
(Kab tak chalega) condition
update

```
for (let i = 1; i < 5; i++) {
  console.log("hello");
}
```

5

(5) $5 < 5 \rightarrow \text{false}$

iteration

Dry Run

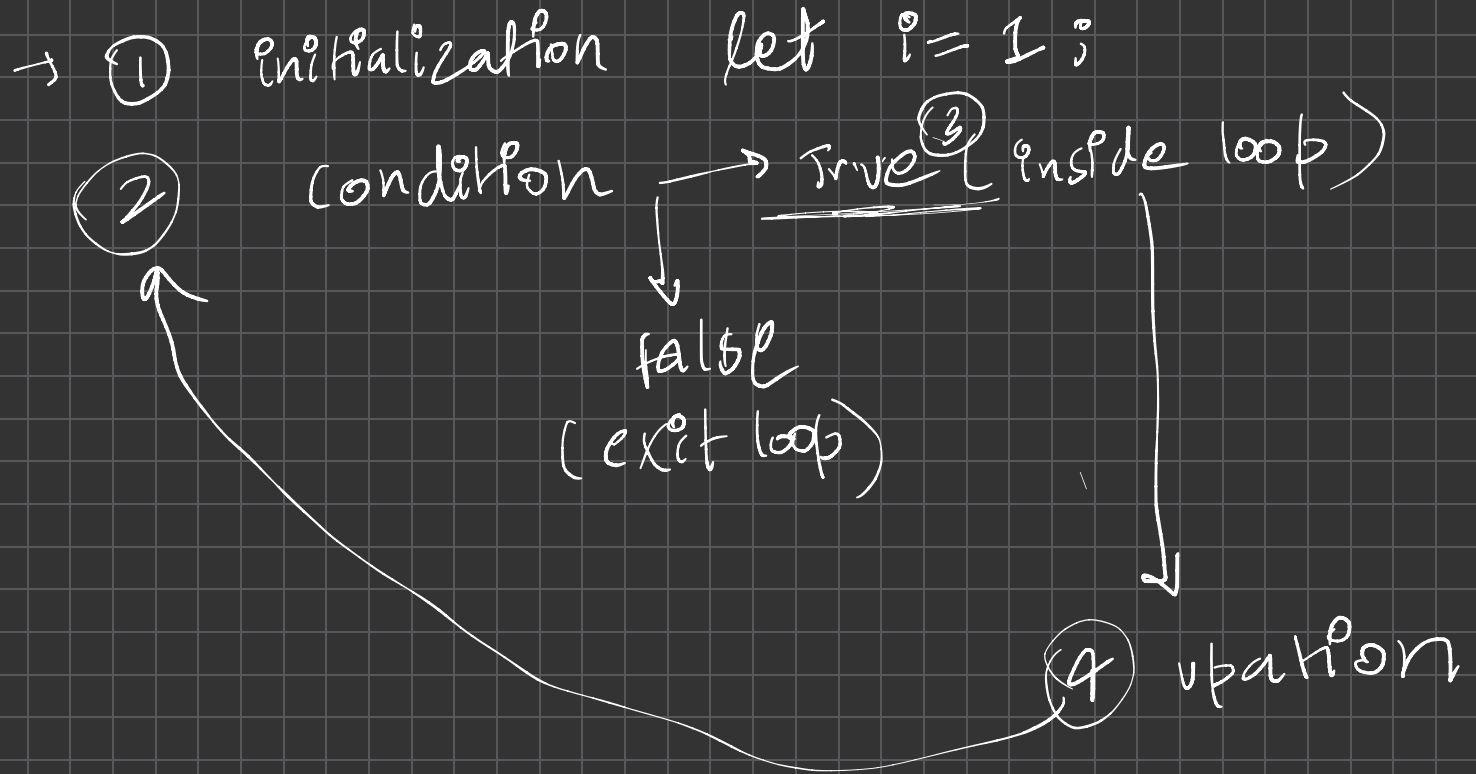
$\frac{2}{1} \frac{3}{1} \frac{4}{1} \rightarrow 5$

① $\rightarrow i < 5 \rightarrow 1 < 5 \rightarrow \text{True}$
"hello"
 $i++ \rightarrow 2$

② $\rightarrow i < 5 \rightarrow 2 < 5 \rightarrow \text{True}$
"hello"
 $i++ \rightarrow 3$

③ $\rightarrow 3 < 5 \rightarrow \text{True}$
"hello", $i++ \rightarrow 4$

④ $\rightarrow 4 < 5 \rightarrow \text{True}$
"hello" $\rightarrow i++ \rightarrow 5$



while
↳ exit nhi pata
work on condition

for loop
tum ko pata
hota hai?
Kaha rukna hai?

① initialization
② condition
③ block
④ updation

```
① let i = 1;  
  while (i <= 5) {  
    console.log(i);  
    i++;  
  }
```

star
X pattern

do {

→ ek bar jarur
chalega

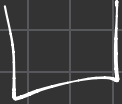
} while (condition)

```
3 ✓ let i = 1;
```

```
4  
5 ① do { ↓  
6     console.log(i);  
7     i++; ②  
8 } while (false) |
```

③ true
↓
false exit

break & continue → skip & continue
terminate
↳ stop & exit
- bucket
apple



Functions

↳ Reusable
Code

keyword

function name

function definition
declaration

```
function totalMarks() {
```

```
}
```

```
function totalMarks(){  
  console.log("Hii");  
}
```

→ declaration

→ function code

totalMarks() → call

Arguments & Parameters

function

funName(¹para1, ²para2, ...) {

para1 → 1

para2 → 2

}

funName(¹arg1, arg2, ...)

value

Default parameter

```
function greetingMsg(userName = "Guest", greetings = "Hi"){  
  // console.log(`${greetings}, ${userName}`);  
  console.log(`${greetings}, ${userName}`);  
  // console.log(greetings + " " + userName);  
}
```

```
greetingMsg("Priyanshu")  
greetingMsg("Anshika" )  
greetingMsg("Satyam" , "Ki hal hai")  
greetingMsg("Saif")
```

return

↪ function

↪ Total Marks

Percentage

```
function totalMarks( mathMarks, scienceMarks, sanskritMarks) {  
    return mathMarks + scienceMarks + sanskritMarks;  
}  
  
function calPercentage(studentName, mathMarks, scienceMarks, sanskritMarks){  
    let total = totalMarks(mathMarks, scienceMarks, sanskritMarks);  
    let percentage = (total / 300) * 100  
    console.log(`${studentName} percentage : ` , percentage);  
}  
  
calPercentage("Alok", 46, 56, 23)
```

①

total Number

```
function totalMarks( mathMarks, scienceMarks, sanskritMarks) {  
  return mathMarks + scienceMarks + sanskritMarks;  → print X  
}
```

②

```
function calPercentage(studentName, mathMarks, scienceMarks, sanskritMarks){  
  let total = totalMarks(mathMarks, scienceMarks, sanskritMarks);  
  let percentage = (total / 300) * 100  
  console.log(`${studentName} percentage : ` , percentage);  
}
```

③

```
calPercentage("Alok", 46, 56, 23)
```



```
console.log("Nishant");
```

Nishant

undefined

- function expression
- arrow function

```
function   funName ( ) {  
  
}
```

```
let funName = function () {  
  
}
```

→ Hoisting Interview

function declaration

vs Expression

```
fun1()  
function fun1(){  
  console.log("function declaration");  
}
```

↪ here you can
use before declaration
(because of hoisting)

```
console.log(add(5, 7));  
let add = function (num1, num2) {  
  return num1 + num2  
}
```

↪ it gives error,
you cannot
use before
initialization in
fun Expression

Arrow function

const fun = () => {

}

no parenthesis needed

single para

return

```
// syntax 1
```

```
let add = num1 => num1 + 4;
```



```
// syntax 2
```

```
let add = (num1, num2) => num1 + num2;
```

```
// syntax 3
```

```
let add = (num1, num2) => {
```

```
  // something
```

```
  // something
```

```
  return num1 + num2
```

```
};
```

By default Return

multiple lines of code
then you have to
mentioned return
explicitly